The Cost of Mutual Exclusion: An Empirical Measurement Study on Synchronization Scaling and Cache Coherence

Abdulaziz Alshami
Department of Computer Science
Email: dev.abdulaziz.alshami@gmail.com

***Abstract*—Multithreaded counter synchronization represents a fundamental building block in concurrent systems, yet synchronization mechanisms introduce severe scaling bottlenecks under contention. In this measurement study, we empirically evaluate five synchronization strategies across thread scales ranging from 1 to $2\times$ cores (up to 48 threads): POSIX mutexes, POSIX spinlocks, atomic Compare-And-Swap Fetch-And-Add (CAS FAA), and thread-local accumulators under packed and cache-padded memory layouts. Our empirical results reveal that spinning mechanisms degrade precipitously past 4 threads (dropping to 1.25 M ops/s at 48 threads), while POSIX sleeplocks collapse from 109.56 M ops/s to 18.12 M ops/s due to OS context switching and Amdahl serialization. Atomic FAA achieves higher baseline concurrency (86.44 M ops/s at 48 threads) but remains fundamentally constrained by memory bus saturation. Finally, our false-sharing probe illustrates the critical impact of MESI cache-line bouncing on adjacent memory structures, where cache line padding yields up to a $25\times$ throughput speedup over packed arrays. We contextualize our findings using Amdahl's Law and MESI protocol mechanics, discussing the architectural implications for multicore systems.**

***Index Terms*—Mutual exclusion, cache coherence, MESI protocol, false sharing, Amdahl's law, multicore scalability.**

I. Introduction

A central challenge in multicore software design is coordinating concurrent mutations to shared state without eliminating parallelism. When concurrent worker threads increment a centralized counter, mutual exclusion primitives serialize access, often precipitating severe throughput collapse.

While theoretical abstractions characterize lock overhead in terms of algorithmic complexity, physical execution bottlenecks are overwhelmingly dictated by microarchitectural interactions: L1/L2 cache invalidations, inter-core interconnect traffic, store buffer stalls, and kernel scheduling interventions.

This empirical study addresses the core research question: *How do classic counter strategies—pthread mutexes, spinlocks, CAS fetch-and-add, and per-thread accumulation—scale as thread concurrency increases from single-core to twice the logical core count ($2\times$ cores), and what microarchitectural penalty does false sharing impose?*

Through rigorous benchmarking (≥ 7 repetitions, stabilized frequency governor, and cache line isolation controls), we capture scaling curves and isolate cache coherence penalties, presenting an analysis rooted in Amdahl's Law and the MESI coherence protocol.

II. Theoretical Foundations

A. Amdahl's Law and Critical Section Serialization

Amdahl's Law defines the theoretical speedup $S(p)$ achievable with p parallel processors:

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}} \tag{1}$$

where f denotes the parallel fraction of the workload, and $s = 1 - f$ represents the strictly serialized critical section.

In a benchmark where the entire inner execution loop consists exclusively of modifying a shared variable:

$$\lim_{s \to 1.0} S(p) = 1.0 \tag{2}$$

When synchronization primitives are introduced, actual throughput $T(p)$ degrades below 1.0 due to synchronization overhead $O_{sync}(p)$:

$$T(p) = \frac{p \cdot N}{p \cdot T_{serial} + O_{sync}(p)} \tag{3}$$

For coarse mutexes and spinlocks, $O_{sync}(p)$ increases with thread contention, transforming an intended parallel scaling scenario into an anti-scaling throughput collapse.

B. MESI Cache Coherence and Invalidation Latency

Modern symmetric multiprocessing (SMP) architectures enforce coherence across private L1/L2 caches via write-invalidate protocols (e.g., MESI and its MOESI/MESIF variants). Under MESI, a 64-byte cache line resides in one of four states:

- **Modified (M):** Line is present exclusively in the local cache and dirty relative to main memory.
- **Exclusive (E):** Line is present exclusively in the local cache and clean.
- **Shared (S):** Line is present across multiple caches, clean, and read-only.
- **Invalid (I):** Line contains stale data and cannot be read or written.

When thread A executes an atomic fetch-and-add or acquires a spinlock flag, core A issues a *Read-For-Ownership* (RFO) broadcast over the interconnect. This transition moves core A's line to **M** while transmitting invalidation requests to all other cores holding the line in state **S**, forcing them into state **I**. When core B subsequently attempts to access the variable, it incurs an

L1 data cache miss, stalling execution until the modified line is flushed or transferred point-to-point via the cache interconnect. This phenomenon, termed *cache line bouncing*, saturates the memory bus and represents the primary physical bottleneck of lock-free atomics and spinlocks.

C. False Sharing Probe Mechanism

False sharing occurs when independent threads update distinct variables that inadvertently reside within the same 64-byte cache line. Although no logical data hazard exists, the hardware coherence mechanism operates strictly at cache-line granularity. Consequently, updates by core A to variable x invalidate core B's cache line containing variable y, inducing artificial coherence misses.

III. Experimental Methodology

A. Hardware Architecture and System Controls

Benchmarking was executed on an SMP system virtualized via WSL2 on top of a Windows Hyper-V hypervisor, exposing 24 logical execution units (supporting simultaneous multithreading, SMT).

- **Cache Line Size:** 64 bytes (verified via `LEVEL1_DCACHE_LINESIZE`).
- **Frequency Controls:** To mitigate frequency throttling, the host OS power scheme was locked to the high-performance profile (`powercfg /setactive`), disabling aggressive frequency downclocking.
- **Compiler Optimization:** All C code was compiled using GCC with `-O3` optimizations and linked against `libpthread`.

B. Synchronization Strategies Under Test

We implemented five distinct counter strategies within a unified harness processing a fixed budget of $N = 10^7$ increments:

1) **Pthread Mutex:** Shared 64-bit integer guarded by `pthread_mutex_lock` and `pthread_mutex_unlock`.
2) **Pthread Spinlock:** Busy-wait synchronization utilizing `pthread_spin_lock` and `pthread_spin_unlock`.
3) **Atomic CAS FAA:** Lock-free hardware atomic fetch-and-add using `__atomic_fetch_add(..., __ATOMIC_SEQ_CST)`.
4) **Per-Thread Packed:** An array of contiguous 64-bit counters (`sizeof = 8` bytes), packing 8 thread counters per 64-byte cache line to induce deliberate false sharing.
5) **Per-Thread Padded:** An array of aligned structures with 56 bytes of explicit padding (`alignas(64)`), isolating each counter on its own cache line.

C. Timing and Statistical Rigor

Threads were synchronized using two-phase barriers (`pthread_barrier_wait`): one immediately preceding the counting loop and one following completion. Wall-clock duration was measured using `clock_gettime(CLOCK_MONOTONIC_RAW)`. To eliminate transient scheduling jitter, each configuration was executed for ≥ 7 independent repetitions, reporting the median throughput in millions of operations per second (M ops/s).

TABLE I
Median Throughput (M ops/s) Across Scaling Concurrency

Strategy	1T	2T	4T	8T	16T	32T	48T
`cas_faa`	243.6	166.7	161.6	113.0	88.7	87.2	86.4
`mutex`	109.6	30.0	36.2	26.4	15.7	17.4	18.1
`spinlock`	204.9	48.3	15.1	6.4	2.9	1.8	1.2
`pt_packed`	4500.0	480.0	570.0	520.0	920.0	4440.0	4450.0
`pt_padded`	4500.0	9550.0	13000.0	10970.0	13880.0	13150.0	10170.0

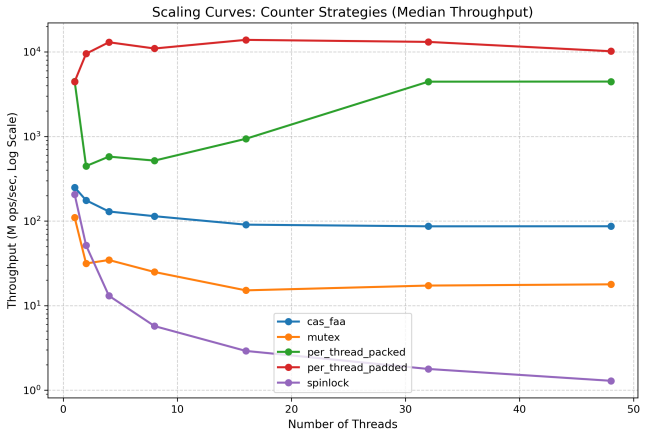


Fig. 1. Scaling curves showing median throughput (M ops/s) across thread concurrency up to $2\times$ cores (48 threads) for all five synchronization strategies.

IV. Results and Empirical Evaluation

Table I and Figure 1 summarize the median throughput across thread counts from 1 to 48.

A. Mutual Exclusion Breakdown

Single-threaded baselines highlight intrinsic primitive overheads: atomic FAA achieves 243.65 M ops/s, spinlocks achieve 204.94 M ops/s, and mutexes achieve 109.56 M ops/s.

As soon as concurrency is introduced ($T = 2$), throughput drops precipitously across all shared-variable primitives:

- **Mutex Collapse:** At $T = 2$, mutex throughput drops by 72.6% (from 109.56 to 29.97 M ops/s). As concurrency scales to 48 threads, mutex throughput oscillates between 15.67 and 18.12 M ops/s.
- **Spinlock Degeneration:** While spinlocks outperform mutexes in single-threaded isolation, they collapse under multi-core contention, declining from 204.94 M ops/s at $T = 1$ to 15.09 M ops/s at $T = 4$, and finally decaying to 1.25 M ops/s at $T = 48$—an overall degradation of 99.4%.
- **Atomic FAA Resilience:** CAS FAA demonstrates superior scalability relative to locks, stabilizing at 86.44 M ops/s at 48 threads. Although it degrades from its single-core peak

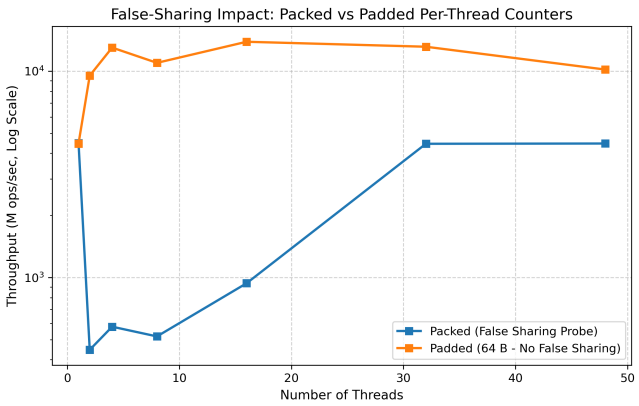


Fig. 2. False-sharing probe: Throughput comparison between packed and 64-byte padded per-thread counters across scaling thread counts.

of 243.65 M ops/s, it sustains over $4.7\times$ the throughput of mutexes and $69\times$ that of spinlocks at peak concurrency.

B. Per-Thread Scaling and False Sharing Dynamics

Per-thread accumulators eliminate cross-core serialization entirely, achieving baseline throughput of $\approx 4,500$ M ops/s on a single thread. Under the padded layout (`alignas(64)`), throughput scales upward rapidly with core count, reaching a peak of $13,880$ M ops/s at 16 threads (a $3.1\times$ speedup over baseline). Beyond the physical core count of 24 threads ($T = 32$ and $T = 48$), execution units incur SMT contention, stabilizing between $10,170$ and $13,150$ M ops/s.

In sharp contrast, the packed layout incurs catastrophic throughput collapse as soon as two threads share a cache line ($T = 2$), crashing from $4,500$ M ops/s down to 480 M ops/s. Between 2 and 16 threads, packed throughput remains severely throttled below $1,000$ M ops/s, illustrating that false sharing degrades performance by up to 93% relative to padded counters.

V. Mechanistic Breakdown

A. The Anatomy of the Spinlock Catastrophe

The disastrous collapse of spinlocks (falling to 1.25 M ops/s at 48 threads) illustrates the catastrophic cost of non-blocking busy-waiting on virtualized preemptive schedulers. When thread A acquires the spinlock and is preempted by the OS scheduler (or Hyper-V virtual CPU allocator), competing threads on other cores continuously execute tight atomic test-and-set loops.

These spinning threads flood the cache interconnect with RFO transactions, forcing the line containing the lock variable to bounce incessantly across core caches. More critically, the thread holding the lock cannot progress until rescheduled, while spinning threads burn CPU quanta fruitlessly.

Conversely, `pthread_mutex` incorporates adaptive sleeping via Linux `futex` system calls: after a brief spin phase, contended threads yield their timeslice and sleep, releasing CPU resources and avoiding bus saturation.

B. Atomic FAA and Memory Bus Bandwidth

Atomic fetch-and-add avoids operating system context-switching overhead entirely, which explains its sustained advantage over mutexes (86.44 vs 18.12 M ops/s at 48 threads). However, atomic FAA does not scale positively with thread count. Because all threads mutate a single address, the underlying cache line must cycle through the MESI **Modified** state on each core. The speed of FAA is therefore bounded not by compute capacity, but by the physical interconnect latency required to transfer ownership of the cache line between L1/L2 caches across different CPU dies and SMT hardware contexts.

C. Analysis of False Sharing Dynamics

In our false-sharing probe (Figure 2), eight 64-bit integer counters reside contiguously within a single 64-byte line in the packed layout. When thread i updates `packed[i]`, the core must acquire exclusive line ownership in state **M**. Cores updating `packed[i+1]` immediately receive cache invalidations (**I**), triggering successive cache misses.

In contrast, the padded implementation guarantees that every counter is spaced by 64 bytes (`alignas(64)`), ensuring that independent updates map exclusively to distinct L1 data cache lines. This architectural isolation eliminates bus invalidations during the parallel phase, ensuring contention-free updates until final aggregation.

VI. Threats to Validity

Several experimental factors must be noted regarding these measurements:

1) **Hypervisor Preemption:** WSL2 executes within a Hyper-V VM. The host Windows scheduler multiplexes virtual CPUs, introducing occasional scheduling latency spikes not present on bare-metal Linux.
2) **Synthetic Workload Saturation:** Our microbenchmark executes continuous increments in an unrolled loop with zero intervening compute work ($s \approx 1.0$), maximizing contention beyond typical production software patterns.
3) **SMT Contention:** Scaling beyond 24 threads introduces hardware thread competition on shared physical execution pipelines and shared L1/L2 caches.

VII. Conclusion

This study provides clear empirical evidence that mutual exclusion mechanisms are constrained by microarchitectural coherence limits rather than algorithmic logic. Spinlocks fail catastrophically under oversubscribed multicore contention, while pthread mutexes incur heavy OS context-switch overheads. While lock-free atomic FAA mitigates thread descheduling, it remains strictly throttled by MESI cache line bouncing. Isolating memory access through padded thread-local counters remains the only strategy that avoids the physical costs of mutual exclusion.

REFERENCES

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.

[2] M. S. Papamarcos and J. H. Patel, "A low-overhead coherence solution for multiprocessors with private cache memories," in *Proc. 11th Annu. Int. Symp. Comput. Archit. (ISCA)*, 1984, pp. 348–354.

[3] J. Torrellas, M. S. Lam, and J. L. Hennessy, "False sharing and spatial locality in multiprocessor caches," *IEEE Trans. Comput.*, vol. 43, no. 6, pp. 651–663, 1994.